

Dataset Overview

For this project, I am going to build and use a Smart Agriculture dataset that records environmental and weather data for various farm plots and areas. Because I do not have physical IoT sensors, I will generate a realistic, structured JSON dataset designed specifically to utilize MongoDB's advanced querying features.

The database will consist of two primary collections. The **Users** collection will handle authentication and store notification preferences (e.g., SMS or email alerts). The **Farms** collection will represent individual farm plots and will include:

- **Core Data:** farm_id, farm_name, owner_id, and crop_type.
- **Spatial Data:** An address sub-document and a location object utilizing exact **GeoJSON coordinates** for precise mapping.
- **Sub-documents:** The sensors array will track active IoT devices and their historical readings. The weather_logs array will store snapshots of local weather conditions, and an alerts_history array will log any severe weather warnings issued for that location.

Application Domain

The application domain is **AgriGuide**. The primary goal is to provide a backend management system that helps farmers monitor crop health, optimize irrigation, track weather and protect their livelihoods from severe weather.

The system will include several key functionalities:

- A key feature of this system is the **Real-Time Disaster & Weather Alerting**.
- Viewing current soil conditions across different farm locations.
- A smart search feature allowing users to find specific farms by their area name and postcode, or via a geographic radius.
- Fetching live weather data from an external API and saving it directly into the database for future analysis.
- Generating analytics and reports that compare historical weather trends against average soil moisture.

By continuously tracking both live weather data and the farm's exact GPS coordinates, the system can automatically notify farmers of sudden weather changes (like unexpected frost) or impending natural disasters (like floods or wildfires) in their specific area. This allows agricultural businesses to take immediate, data-driven preventative actions to protect their crops and conserve resources. This system will benefit agricultural businesses by allowing them to

make data-driven decisions regarding irrigation, ultimately improving crop yields, increasing awareness and conserving resources.

Implementation Plan

The backend system will be built using Python Flask and will include several RESTful API endpoints. To demonstrate advanced database functionality, the API will go beyond basic CRUD operations to include complex geospatial queries, aggregation pipelines, compound text searches, and external API integration.

- **POST /farms:** Create a new farm record.
- **GET /farms/<id>:** Get detailed information about a single, specific farm using its unique ID.
- **PUT /farms/<id>:** Update top-level farm data (e.g., changing the crop type).
- **DELETE /farms/<id>:** Remove a farm record entirely.
- **Complex Search (GET /farms/search):** Search for farms using a combination of `area_name` and `postcode` using MongoDB compound indexes.
- **External Integration (POST /farms/<id>/sync_weather):** Fetch live weather data from an external API based on the farm's coordinates and push it into the database.
- **Data Aggregation (GET /farms/analytics):** Utilize MongoDB aggregation pipelines to analyze past sensor readings alongside weather trends.
- **Geospatial Disaster Alerting (POST /alerts/broadcast):** An endpoint that accepts a GeoJSON polygon representing a disaster zone, queries the database to find all farms intersecting that area, and generates warning notifications for the affected owners.
- **GET /farms:** Retrieve public information on all registered farms.
- **POST /farms/<id>/sensors:** Add a new sensor sub-document to a farm.

User Roles & Access Control:

- **Admin:** Full CRUD access to view, add, modify, or delete any farm record, and the authority to broadcast emergency disaster alerts to specific geographic regions.
- **Authenticated User (Farmer):** Can manage their specific farm profiles, sync local weather data, configure their notification preferences, and receive targeted alerts if their farm falls within a danger zone.
- **Guest (Unauthenticated):** Restricted access to view public, aggregated crop data and utilize basic search functions.